

NER-SMARTLOG | Code-Level Technical Q&A Guide

Line-by-Line Python & HTML/JS Code Analysis for Hackathon / Viva Defense

Smart India Hackathon 2026 | Problem Statement: 26002 | Ministry of Development of North Eastern Region

SECTION 1: PYTHON BACKEND CODE ANALYSIS (app.py)

1. Flask App Initialization & ChoiceLoader (Lines 1-18)

Q1: Why did you use 'jinja2.ChoiceLoader' instead of standard Flask template lookup?

Ans: Standard Flask only looks in a hardcoded 'templates/' subfolder. If the project is executed from different parent directories or has index.html in the root, Jinja throws TemplateNotFound. ChoiceLoader chains FileSystemLoader([BASE_DIR/templates,] Tj T* (BASE_DIR, '.']), ensuring 100% template availability.

2. Gazetteer & Geocoding Logic (Lines 74-125)

Q2: How does the 'geocode_place(query)' function work and why is it resilient?

Ans: It uses a 33 pre-indexed Himalayan hubs like Tawang,

If both fail/timeout, it gracefully defaults to Guwahati Hub (26.1445, 91.7362), preventing server crashes.

3. Open-Meteo Weather Telemetry (Lines 185-235)

Q3: Explain the 'get_8_states_weather()' function and how it classifies risk.

Ans: It queries Open-Meteo NWP forecast API for all 8 state capital coordinates (current temp, rain, wind_speed, humidity). WMO weather codes (0=Clear, 51-80=Rain, 95+=Thunderstorm) map to conditions. Risk is dynamically assigned: HIGH if rain > 18mm or high landslide-prone zone (Arunachal), MEDIUM if rain > 5mm, else LOW. Offline fallbacks ensure continuous UI feed.

4. Multi-Factor Risk Calculation Engine (Lines 237-242)

Q4: Explain the weights and formula in 'calculate_risk()'.

Ans: Formula: Score = (0.30*Rain) + (0.25*Landslide) + (0.20*Road) + (0.10*Traffic) + (0.10*Historical) + (0.05*TravelTime). Precipitation (0.30) and Landslide slope (0.25) have the highest weight because monsoonal rainfall on steep hills is the primary trigger for road washouts in the North East. Thresholds: <=30: LOW, <=60: MEDIUM, <=80: HIGH, >80: CRITICAL.

SECTION 2: PYTHON API ROUTING & BACKEND LOGIC (app.py)

5. Route Optimization Endpoint '/api/find-routes' (Lines 245-345)

Q5: How does '/api/find-routes' handle routing and what is the fallback mechanism?

'min(generated_routes, key=lambda r: r["score"])', returning JSON with AI recommendation summary.

6. Vehicle Tracking & AI Chatbot Endpoints (Lines 347-410)

Q6: How do '/api/track-vehicle' and '/api/chat' operate?

Ans: '/api/track-vehicle' searches the vehicles dataset by registration number (NER-MED-01, NER-OXY-09). If not found, it generates a live simulated convoy profile to guarantee uninterrupted UX.

'/api/chat' performs semantic keyword extraction ('from...to', 'tawang', 'weather', 'track'), querying real-time backend state and returning formatted markdown string advisories in under 20ms.

7. Disaster Reporting & Session Controllers (Lines 412-465)

Q7: Explain the data flow in '/submit-disaster-report' and session authentication.

Ans: '/submit-disaster-report' parses form inputs, assigns a unique incident ID ('DIS-' + len), sets coordinates, and prepends the incident to the incidents list so the newest hazard appears at the top. It then redirects to '/'.

'/login' establishes cryptographically signed session cookies ('session["user"]', 'session["role"]').

'/' checks 'user not in session' -> redirects to login, otherwise renders index.html with live weather and incident data.

SECTION 3: HTML, CSS & JAVASCRIPT FRONTEND ANALYSIS (index.html)

8. Single-File Unified View Architecture (Lines 40-160)

Q8: How does index.html toggle between Login and Dashboard views in a single file?

Ans: Jinja2 conditional inline styles control display: '#auth-view' uses 'display: {% if is_login or not user %}flex{% else %}none{% endif %}', while '#dashboard-view' uses 'display: {% if not is_login and user %}flex{% else %}none{% endif %}'. This eliminates multi-file template fragmentation and allows seamless state transitions.

9. Leaflet.js GIS Mapping & Telemetry Layers (Lines 530-620)

Q9: How are map layers, alternative polylines, and convoy animations rendered in JS?

draws alternative polylines (Green weight 6 for AI Best Route, Blue weight 4 for alternatives), and calls 'map.flyToBounds(featureGroup.getBounds(), {padding: [40, 40]})' for smooth cinematic bounding.

[v.lat, v.lon], 11, {duration: 1.5}

10. 1-Click Judge Instant Login Function (Lines 505-515)

Q10: How does the 'quickDemoLogin(user, role)' JavaScript function work?

Ans: It dynamically populates the login form fields ('login-username', 'login-password', 'login-role') with pre-authenticated demo credentials and immediately triggers 'form.submit()', enabling hackathon judges to test any role with 1 click in under 1 second.

SECTION 4: RAPID VIVA SUMMARY FOR JUDGES

â€ Frontend Tech : Leaflet.js 1.9.4, HTML5, CSS3 Variables, Asynchronous Fetch API, Glassmorphic UI.

â€ Backend Tech : Python 3.13, Flask 3.x, Jinja2 ChoiceLoader, Open-Meteo REST API, OSRM Routing Engine.

â€ Key Innovation: Predictive Hazard Cost Scoring (Rain + Slope + BRO Alert) vs. Reactive Commercial Traffic Speed.